

Space Explorer Dashboard — Debugging Notes

Concepts and mistakes covered while fixing index.js

1. Case-sensitive DOM properties

Concept

JavaScript's built-in DOM property is **classList** (capital L). JavaScript is case-sensitive, so any other casing is treated as a completely different, undefined property.

Mistake made

Wrote **classlist** (lowercase l) instead of **classList**.

Before

```
navLinks[i].classlist.remove("bg-blue-500/10", "text-blue-400");
```

After

```
navLinks[i].classList.remove("bg-blue-500/10", "text-blue-400");
```

Why it matters

element.classlist is **undefined**, so calling **.remove()** on it throws "Cannot read properties of undefined" and stops the whole function right there.

2. Always initialize your loop counter

Concept

A **for** loop needs all three parts: start value, condition, and step (**for (var i = 0; i < n; i++)**). Skipping the start value leaves **i** as **undefined**.

Mistake made

Wrote **for(var i ; i < navLinks.length; i++)** — no starting value for **i**.

Before

```
for(var i ; i < navLinks.length; i++){ ... }
```

After

```
for(var i = 0; i < navLinks.length; i++){ ... }
```

Why it matters

undefined < navLinks.length evaluates to **false**, so the loop body never runs at all — not even once. Bugs like this fail silently, with no error thrown.

3. Exact string matching for attributes

Concept

getAttribute(name) and **data-*** attributes require the exact attribute name, character for character — including no stray whitespace.

Mistake made

Wrote `e.target.getAttribute("data-section ")` — with a trailing space inside the string.

Before

```
if (e.target.getAttribute("data-section ") === "planets") { ... }
```

After

```
if (e.target.getAttribute("data-section") === "planets") { ... }
```

Why it matters

The real attribute is **data-section** (no space), so this always returned **null** and none of the if/else branches ever matched — nothing happened on click, with no error.

4. Event delegation for dynamic content

Concept

When elements are created and destroyed dynamically (e.g. rebuilt via **innerHTML**), attach one listener to a stable parent that always exists, and use `e.target.closest(selector)` inside the handler to figure out what was actually clicked. A listener attached directly to a dynamic element is destroyed along with it.

Mistake made

Attached listeners directly to buttons (**load-date-btn**, **today-apod-btn**) that live inside a container rebuilt by **innerHTML** — so the listeners vanished on every re-render, and also didn't exist yet at page-load time when the listener was first attached.

Before

```
var loadDataBtn = document.getElementById("load-date-btn");
loadDataBtn.addEventListener("click", ...); // crashes: element not created yet
```

After

```
todaySection.addEventListener("click", function (e) {
  if (e.target.closest("#load-date-btn")) {
    // handle load click
  }
});
```

Why it matters

Delegation survives content being rebuilt, because the listener lives on the permanent parent, not on the element that keeps getting deleted and recreated.

5. Await before you render — async race conditions

Concept

async functions don't pause the rest of your code — calling one without **await** just starts it in the background and immediately continues to the next line, before the network request has finished.

Mistake made

Called **showAnyDayInSpace()** immediately after starting **AnyDayInSpaceApi(...)**, instead of waiting for the fetch to actually complete.

Before

```
AnyDayInSpaceApi(input.value); // starts a fetch, does not wait
showAnyDayInSpace();           // runs immediately, using stale/old data
```

After

```
async function AnyDayInSpaceApi(dateValue) {
  var res = await fetch(...);
  var dataFromApi = await res.json();
  anyDayInSpaceFromApi = dataFromApi;
  showAnyDayInSpace(); // now runs only once data has really arrived
}
```

Why it matters

The UI would render using whatever data happened to already be sitting in the variable — sometimes stale, sometimes empty — which is exactly why the bug appeared to work "sometimes" and not others.

6. Check `res.ok` before calling `res.json()`

Concept

`fetch()` does not throw an error for HTTP error statuses (404, 502, 503, etc.) — it only rejects on network failure. A failed request still resolves successfully, but its body may be an HTML error page instead of JSON.

Mistake made

Called `res.json()` unconditionally, assuming every response was valid JSON.

Before

```
var res = await fetch(url);
var dataFromApi = await res.json(); // crashes if the server sent back HTML/text
```

After

```
var res = await fetch(url);
if (!res.ok) {
  console.error("API error:", res.status);
  return;
}
var dataFromApi = await res.json();
```

Why it matters

When the server fails and returns an HTML error page, `res.json()` tries to parse "<html>..." as JSON and throws **"Unexpected token '<'"** — an uncaught, confusing error that a simple status check avoids entirely.

7. IDs must be unique — use classes inside loops

Concept

`id` attributes must be unique per page. `document.getElementById(id)` silently returns only the **first** matching element if there are duplicates — it never errors, and never returns the rest.

Mistake made

Gave every `<img>` generated inside a loop the exact same `id="allLaunch-image"`, then used `getElementById` to attach an error handler.

Before

```
    <!-- repeated in every loop iteration -->

var img = document.getElementById("allLaunch-image");
img.addEventListener("error", ...);    // only ever attaches to the FIRST image
```

After

```


var images = document.querySelectorAll(".all-launch-image");
images.forEach(function (img) {
  img.addEventListener("error", function (e) {
    e.target.src = "../assets/images/launch-placeholder.png";
  });
});
```

Why it matters

Only the first card's broken-image fallback ever worked; every other card with a failed image just showed the browser's default broken-image icon, with no listener catching it.

8. Event names are just strings — typos fail silently

Concept

addEventListener(type, handler) does not validate the event name. Any string is accepted, even one that doesn't correspond to a real DOM event — it simply never fires.

Mistake made

Wrote **addEventListener("clicks", ...)** instead of **"click"**.

Before

```
planetsCard.addEventListener("clicks", function (e) { ... });
```

After

```
planetsCard.addEventListener("click", function (e) { ... });
```

Why it matters

There is no **"clicks"** event, so the handler registers successfully but is never invoked — no error, nothing happens on click, and it's easy to miss since it looks correct at a glance.

9. Guard against null before using a value

Concept

Both **Element.closest(selector)** and API response fields can legitimately be **null** — **closest()** returns null if no ancestor matches, and an API may return null for a field that doesn't apply (e.g. a planet with no moons). Always check before using the value.

Mistake made

Used **target.getAttribute(...)** without checking whether **target** (from **closest()**) was null, and used **planet.moons.length** without checking whether **moons** was null.

Before

```
target = e.target.closest(".planet-card");
if (planetsFromApi[i].englishName... === target.getAttribute("data-planet-id")) // crashes if target is

${planetsFromApi[i].moons.length} // crashes for planets with no moons (moons: null)
```

After

```
var target = e.target.closest(".planet-card");
if (!target) return; // click missed every card - stop here

${planetsFromApi[i].moons ? planetsFromApi[i].moons.length : 0}
```

Why it matters

Mercury and Venus return **"moons": null** from the API (they have no moons), so this crashed specifically — and only — when clicking those two planet cards, making the bug look intermittent.

Recurring theme: almost all of these bugs fail *silently* — no error is thrown (typos in event names, uninitialized loop counters, attribute name mismatches) or the error only shows up under specific conditions (async races, null fields for certain data, duplicate IDs). The fastest way to catch these earlier is to add small guard checks (null checks, `res.ok` checks) and to test the edge cases directly — the first item in a list, the last, and the ones with missing/empty data.